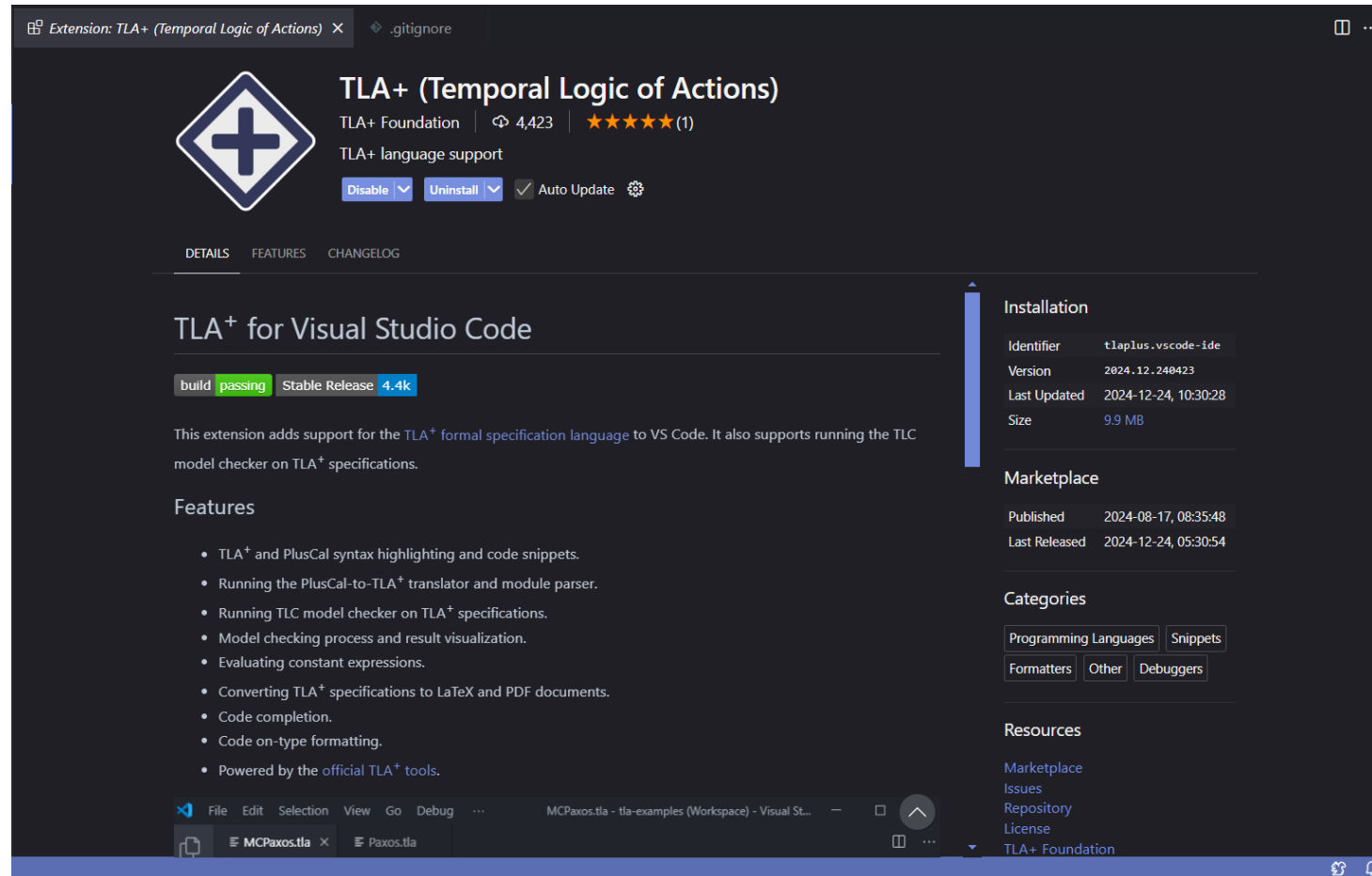




Verifying Car Alarm System with TLA

Sebastian David Albu, Simone Franza
Formal Specification and Design of Software, WS 2024

Tools



Extension: TLA+ (Temporal Logic of Actions) ✕ .gitignore

TLA+ (Temporal Logic of Actions)

TLA+ Foundation | 4,423 | ★★★★★ (1)

TLA+ language support

Disable Uninstall Auto Update

DETAILS FEATURES CHANGELOG

TLA+ for Visual Studio Code

build passing Stable Release 4.4k

This extension adds support for the TLA+ formal specification language to VS Code. It also supports running the TLC model checker on TLA+ specifications.

Features

- TLA+ and PlusCal syntax highlighting and code snippets.
- Running the PlusCal-to-TLA+ translator and module parser.
- Running TLC model checker on TLA+ specifications.
- Model checking process and result visualization.
- Evaluating constant expressions.
- Converting TLA+ specifications to LaTeX and PDF documents.
- Code completion.
- Code on-type formatting.
- Powered by the official TLA+ tools.

Installation

Identifier	tlaplus.vscode-ide
Version	2024.12.240423
Last Updated	2024-12-24, 10:30:28
Size	9.9 MB

Marketplace

Published	2024-08-17, 08:35:48
Last Released	2024-12-24, 05:30:54

Categories

Programming Languages Snippets

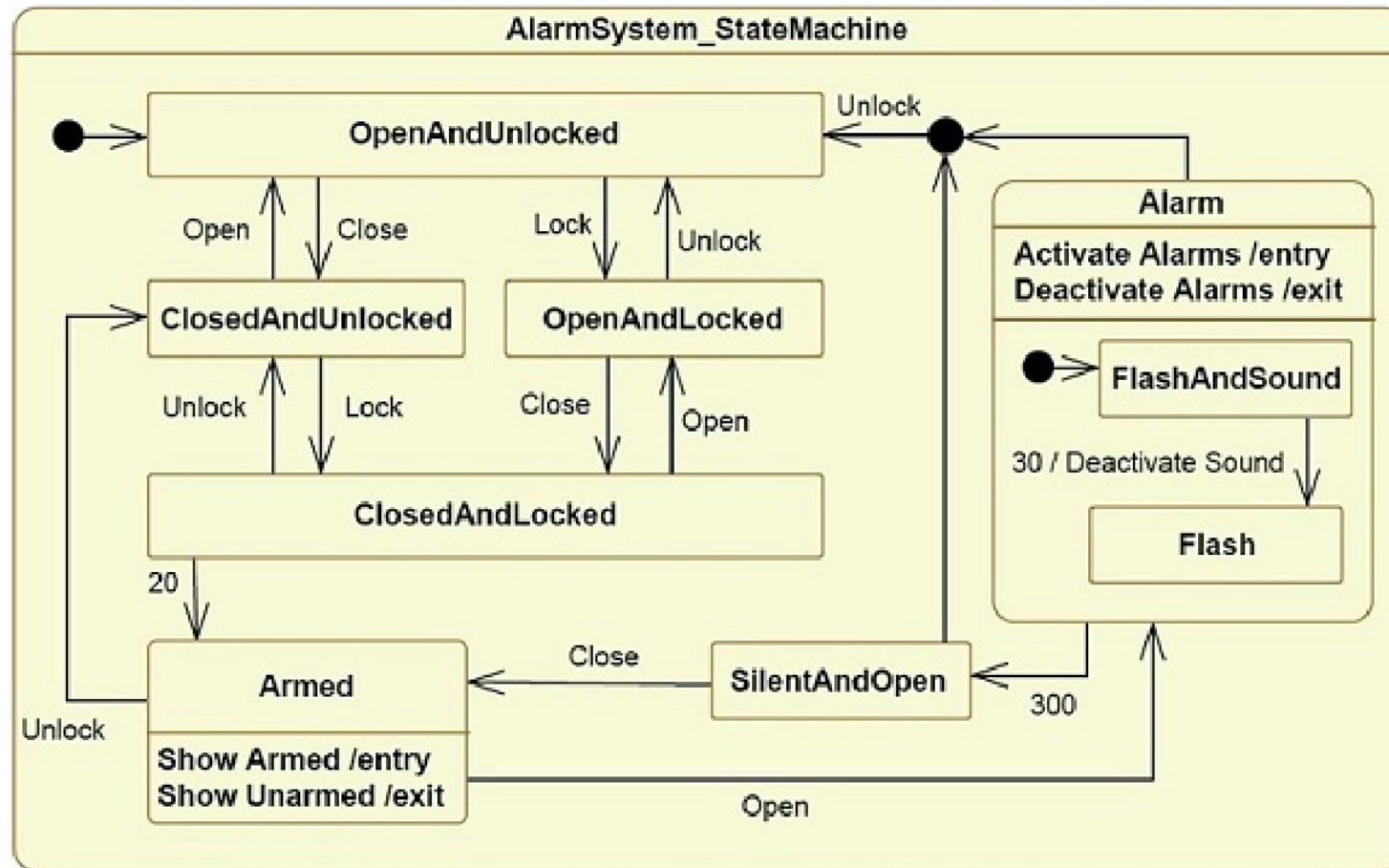
Formatters Other Debuggers

Resources

Marketplace
Issues
Repository
License
TLA+ Foundation



Car Alarm System



Initial Abstraction



Three fundamental requirements:

- Arming
- Alarm
- Deactivation

Key features:

- Opening / Closing
- Unlocking / Locking
- Arming
- Alarm System

Initial Abstraction: Assumptions

Boolean variables & atomic transitions

Components:

- Abstract doors
- Bonnet
- Luggage compartment

No timing logic → non-deterministic actions

- Arming system
- Disable alarm
- Disable flashing lights

Initial Abstraction: Next

Next: Defines the possible transitions from one state to another. The system can evolve by performing one of the actions

$$\begin{aligned} \text{Next} \triangleq & \vee \text{OpenDoorDo} \\ & \vee \text{CloseDoorDo} \\ & \vee \text{OpenBonnetDo} \\ & \vee \text{CloseBonnetDo} \\ & \vee \text{OpenLuggageDo} \\ & \vee \text{CloseLuggageDo} \\ & \vee \text{UnlockLuggageDo} \\ & \vee \text{LockDo} \\ & \vee \text{UnlockDo} \\ & \vee \text{ArmDo} \\ & \vee \text{DisableAlarmDo} \\ & \vee \text{DisableFlashingLightsDo} \end{aligned}$$

Initial Abstraction: Theorems

SECTION 5: Theorems

Type Invariant Theorem: Ensures that the *TypeInvariant* holds in all reachable states.

THEOREM $Spec \Rightarrow \Box TypeInvariant$

Theorem 1: Locked and all doors closed will eventually arm the system.

THEOREM $\wedge locked = TRUE$
 $\wedge all_doors_closed = TRUE$
 $\wedge luggage_compartment_closed = TRUE$
 $\wedge bonnet_closed = TRUE$
 $\Rightarrow \Diamond(armed = TRUE)$

Theorem 2: Armed system with access attempt triggers the alarm.

THEOREM $\wedge armed = TRUE$
 $\wedge \vee all_doors_closed = FALSE$
 $\vee luggage_compartment_closed = FALSE$
 $\vee bonnet_closed = FALSE$
 $\Rightarrow \Diamond(alarm = TRUE)$

Theorem 3: Alarm will deactivate upon unlocking the vehicle.

THEOREM $alarm = TRUE \Rightarrow \Diamond(locked = FALSE \wedge alarm = FALSE)$

Theorem 4: Hazard lights, if active, will eventually turn off.

THEOREM $lights_on = TRUE \Rightarrow \Diamond(lights_on = FALSE)$

Theorem 5: Hazard lights only turn on when the alarm is active.

THEOREM $lights_on = TRUE \Rightarrow alarm = TRUE$

Theorem 6: Armed system remains armed until unlocked.

THEOREM $armed = TRUE \Rightarrow \Box(\wedge (armed = TRUE \Rightarrow locked = TRUE)$
 $\wedge (armed = FALSE \Rightarrow locked = FALSE))$

Theorem 7: If unlocked, the system cannot be armed or alarming.

THEOREM $\Box(locked = FALSE \Rightarrow armed = FALSE \wedge alarm = FALSE)$

Theorem 8: Doors must eventually close after being opened.

THEOREM $all_doors_closed = FALSE \Rightarrow \Diamond(all_doors_closed = TRUE)$

Theorem 9: It can never happen that the system is armed and unlocked.

THEOREM $\neg\Box(armed = TRUE \wedge locked = FALSE)$

Initial Abstraction: Properties



Initial Abstraction: Properties

MODULE *MC_CarAlarmBasic*
EXTENDS *CarAlarmBasic*, *TLC*

P1: If the vehicle is locked and all doors, compartment and bonnet are closed, it eventually becomes armed unless unlocked.

$$\begin{aligned} \text{prop1} &\triangleq \wedge \text{locked} = \text{TRUE} \\ &\quad \wedge \text{all_doors_closed} = \text{TRUE} \\ &\quad \wedge \text{luggage_compartment_closed} = \text{TRUE} \\ &\quad \wedge \text{bonnet_closed} = \text{TRUE} \\ &\quad \leadsto \text{locked} = \text{FALSE} \vee \text{armed} = \text{TRUE} \end{aligned}$$

P2: If the system is armed and any door or compartment or bonnet is opened, the alarm eventually triggers or the vehicle is unlocked.

$$\begin{aligned} \text{prop2} &\triangleq \wedge \text{armed} = \text{TRUE} \\ &\quad \wedge \vee \text{all_doors_closed} = \text{FALSE} \\ &\quad \quad \vee \text{luggage_compartment_closed} = \text{FALSE} \\ &\quad \quad \vee \text{bonnet_closed} = \text{FALSE} \\ &\quad \leadsto \text{locked} = \text{FALSE} \vee \text{alarm} = \text{TRUE} \end{aligned}$$

P3: The vehicle, if locked and with all doors, compartment and bonnet closed, eventually enters the armed state unless unlocked.

$$\begin{aligned} \text{prop3} &\triangleq \wedge \text{locked} = \text{TRUE} \\ &\quad \wedge \text{all_doors_closed} = \text{TRUE} \\ &\quad \wedge \text{luggage_compartment_closed} = \text{TRUE} \\ &\quad \wedge \text{bonnet_closed} = \text{TRUE} \\ &\quad \wedge \Box(\text{locked} = \text{TRUE}) \\ &\quad \Rightarrow \Diamond(\text{armed} = \text{TRUE}) \end{aligned}$$


Initial Abstraction: Properties

MODULE *MC_CarAlarmBasic*
EXTENDS *CarAlarmBasic*, *TLC*

P1: If the vehicle is locked and all doors, compartment and bonnet are closed, it eventually becomes armed unless unlocked.

$$\begin{aligned} \text{prop1} &\triangleq \wedge \text{locked} = \text{TRUE} \\ &\quad \wedge \text{all_doors_closed} = \text{TRUE} \\ &\quad \wedge \text{luggage_compartment_closed} = \text{TRUE} \\ &\quad \wedge \text{bonnet_closed} = \text{TRUE} \\ &\quad \leadsto \text{locked} = \text{FALSE} \vee \text{armed} = \text{TRUE} \end{aligned}$$

P2: If the system is armed and any door or compartment or bonnet is opened, the alarm eventually triggers or the vehicle is unlocked.

$$\begin{aligned} \text{prop2} &\triangleq \wedge \text{armed} = \text{TRUE} \\ &\quad \wedge \vee \text{all_doors_closed} = \text{FALSE} \\ &\quad \quad \vee \text{luggage_compartment_closed} = \text{FALSE} \\ &\quad \quad \vee \text{bonnet_closed} = \text{FALSE} \\ &\quad \leadsto \text{locked} = \text{FALSE} \vee \text{alarm} = \text{TRUE} \end{aligned}$$

P3: The vehicle, if locked and with all doors, compartment and bonnet closed, eventually enters the armed state unless unlocked.

$$\begin{aligned} \text{prop3} &\triangleq \wedge \text{locked} = \text{TRUE} \\ &\quad \wedge \text{all_doors_closed} = \text{TRUE} \\ &\quad \wedge \text{luggage_compartment_closed} = \text{TRUE} \\ &\quad \wedge \text{bonnet_closed} = \text{TRUE} \\ &\quad \wedge \Box(\text{locked} = \text{TRUE}) \\ &\quad \Rightarrow \Diamond(\text{armed} = \text{TRUE}) \end{aligned}$$

P4: When armed, opening any door, compartment or bonnet will eventually activate the alarm, unless disarmed.

$$\begin{aligned} \text{prop4} &\triangleq \wedge \text{armed} = \text{TRUE} \\ &\quad \wedge \vee \text{all_doors_closed} = \text{FALSE} \\ &\quad \quad \vee \text{luggage_compartment_closed} = \text{FALSE} \\ &\quad \quad \vee \text{bonnet_closed} = \text{FALSE} \\ &\quad \wedge \Box(\text{armed} = \text{TRUE}) \\ &\quad \Rightarrow \Diamond(\text{alarm} = \text{TRUE}) \end{aligned}$$

P5: Hazard lights turn off after being active.

$$\text{prop5} \triangleq \text{lights_on} = \text{TRUE} \Rightarrow \Diamond(\text{lights_on} = \text{FALSE})$$

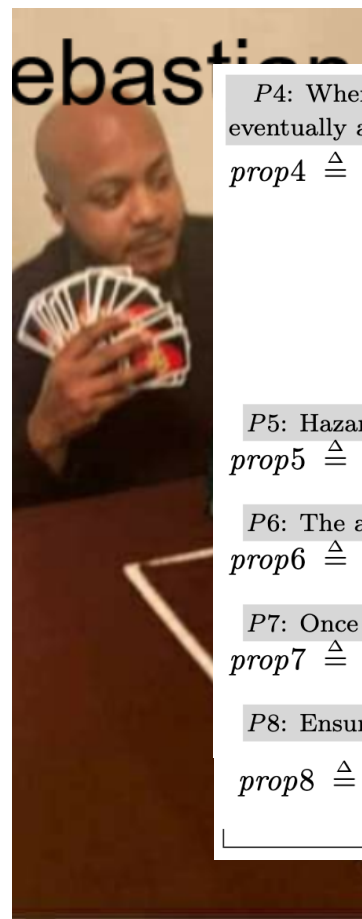
P6: The alarm can always be deactivated by unlocking the vehicle.

$$\text{prop6} \triangleq \text{alarm} = \text{TRUE} \Rightarrow \Diamond(\text{locked} = \text{FALSE} \wedge \text{alarm} = \text{FALSE})$$

P7: Once armed, the system will stay armed until the vehicle is unlocked.

$$\text{prop7} \triangleq \text{armed} = \text{TRUE} \Rightarrow \Box(\text{armed} = \text{TRUE} \Rightarrow \text{locked} = \text{TRUE})$$

P8: Ensure that it is never possible to have the car armed without being locked

$$\text{prop8} \triangleq \neg \Box(\text{armed} = \text{TRUE} \wedge \text{locked} = \text{FALSE})$$


First Refinement: Doors



Two new requirements:

- 2 or 4 doors
- Luggage unlocking

Key features:

- Number of open/closed doors
- Luggage locked

Doors Refinement: Theorems

***** THEOREMS *****

Theorem 1: Ensures that the specification of the original car alarm system holds in this extended version of the system.

THEOREM $Spec \Rightarrow CarAlarmBasic!FairSpec$

Theorem 2: If the vehicle is locked and all doors are closed, it is always possible to unlock the luggage compartment while keeping the system armed.

THEOREM $\wedge locked = TRUE$
 $\wedge closed_doors = num_doors$
 $\wedge open_doors = 0$
 $\wedge \Box(locked = TRUE)$
 $\Rightarrow \Diamond(luggage_compartment_locked = FALSE \wedge armed = TRUE)$

Theorem 3: The system can remain armed while the luggage compartment is unlocked.

THEOREM $locked = TRUE \wedge armed = TRUE$
 $\Rightarrow \Box(luggage_compartment_locked = FALSE \Rightarrow armed = TRUE)$

Theorem 4: Opening a door while armed will eventually trigger the alarm.

THEOREM $armed = TRUE \wedge closed_doors < num_doors \wedge open_doors > 0$
 $\Rightarrow \Diamond(alarm = TRUE)$

Theorem 5: Once all doors are closed, it will eventually be possible to arm the system.

THEOREM $closed_doors = num_doors \wedge open_doors = 0 \wedge locked = TRUE$
 $\Rightarrow \Diamond(armed = TRUE)$

Theorem 6: Hazard lights will turn off after being triggered.

THEOREM $lights_on = TRUE \Rightarrow \Diamond(lights_on = FALSE)$

Theorem 7: Unlocking the luggage compartment while armed will not deactivate the system.

THEOREM $armed = TRUE \wedge luggage_compartment_locked = FALSE \Rightarrow \Box(armed = TRUE)$

Theorem 8: It can never happen that the system is armed and unlocked.

THEOREM $\neg \Box(armed = TRUE \wedge locked = FALSE)$

Theorem 9: It can never happen that the system is unlocked and the luggage locked.

THEOREM $\neg \Box(locked = FALSE \wedge luggage_compartment_locked = TRUE)$

Timers Refinement



Timers:

- Arm delay - 20s
- Alarm - 30s
- Hazard Lights - 300s

Key features:

- Set semantics
- Set for active timers
- Mapping for remaining time

Timers Refinement: Workflow

active_timers:

- Set of active timers

timer:

- Mapping name to value

Predicates:

- Start
- Remove
- Set multiple
- Check if active

Timers added via:

- Opening → alarm, hazard lights
- Closing → armed
- Locking → armed

Timers removed via:

- Expiration
- Opening → armed
- Unlocking → alarm, hazard lights

Timers Refinement: Workflow

$$\begin{aligned}
 \text{Tick}T &\triangleq \\
 &\text{LET } \text{cur_timers} \triangleq \{ \text{timer}[t] : t \in \text{active_timers} \} \\
 &\quad \text{next_time} \triangleq \text{MinTimer}(\text{cur_timers}) \\
 &\quad \text{expired_timers} \triangleq \{ t \in \text{active_timers} : \text{timer}[t] = \text{next_time} \} \\
 &\text{IN} \\
 &\quad \wedge \text{active_timers} \neq \{ \} \\
 &\quad \wedge \text{AllTimersGreaterEq}(\text{next_time}) \\
 &\quad \wedge \text{timer}' = [\\
 &\quad \quad t \in \text{DOMAIN } \text{timer} \\
 &\quad \quad \mapsto \\
 &\quad \quad \text{IF } t \in \text{active_timers} \\
 &\quad \quad \quad \text{THEN} \\
 &\quad \quad \quad \quad \text{timer}[t] - \text{next_time} \\
 &\quad \quad \quad \text{ELSE} \\
 &\quad \quad \quad \quad 0 \\
 &\quad \quad] \\
 &\quad \text{Remove all timers that reached 0} \\
 &\quad \wedge \text{active_timers}' = \{ t \in \text{active_timers} : \text{timer}'[t] > 0 \} \\
 &\quad \wedge \text{Handle actions for specific timers when they reach 0} \\
 &\quad \vee (\forall t \in \text{expired_timers} : \\
 &\quad \quad \vee (t = \text{"armed"} \Rightarrow \text{HandleArmedTimer}) \\
 &\quad \quad \vee (t = \text{"alarm"} \Rightarrow \text{HandleAlarmTimer}) \\
 &\quad \quad \vee (t = \text{"flashing_lights"} \Rightarrow \text{HandleFlashingLightsTimer}))
 \end{aligned}$$

Tick action:

- Minimum remaining time
- Subtract from active timers
- Handle timer expirations

Timers Refinement: Theorems

***** THEOREMS *****

Basic Specification Refinement

THEOREM $SpecT \Rightarrow Spec$ Timing refinement preserves original spec

Timer State Consistency Theorems

THEOREM $isTimerActive("alarm") \equiv ("alarm" \in active_timers)$

THEOREM $isTimerActive("flashing_lights") \equiv ("flashing_lights" \in active_timers)$

THEOREM $isTimerActive("armed") \equiv ("armed" \in active_timers)$

System State Consistency Theorems

THEOREM $alarm \equiv isTimerActive("alarm")$

THEOREM $lights_on \equiv isTimerActive("flashing_lights")$

THEOREM $armed \Rightarrow \neg isTimerActive("armed")$

THEOREM $\neg \Box (armed = TRUE \wedge locked = FALSE)$

THEOREM $\neg \Box (locked = FALSE \wedge luggage_compartment_locked = TRUE)$

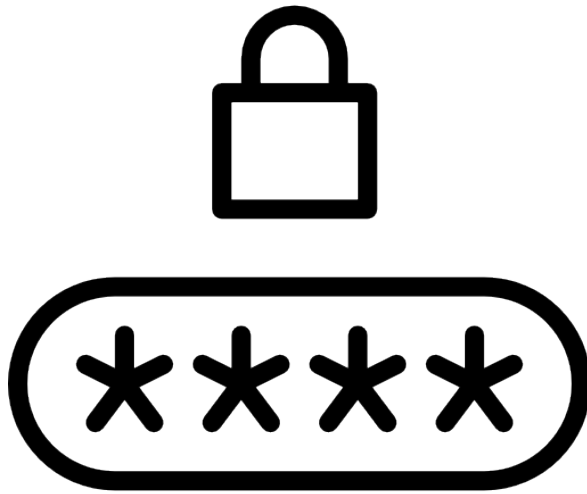
Timer Duration Theorems

THEOREM $isTimerActive("armed") \Rightarrow timer["armed"] \leq max_time_arm$

THEOREM $isTimerActive("alarm") \Rightarrow timer["alarm"] \leq max_time_alarm$

THEOREM $isTimerActive("flashing_lights") \Rightarrow timer["flashing_lights"] \leq max_time_hazard$

PIN Code Refinement



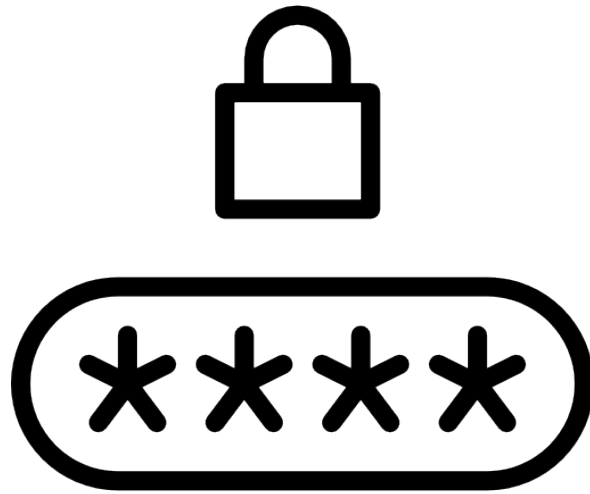
New requirements:

- 4 digit PIN
- SetPinCode

Key features:

- New Unlock action
- Change PIN
- Integrate old actions

PIN Code Refinement



Assumptions:

- Tries reset on unlock
- SetPinCode replaces old PIN
 - ➔ On success removes alarm
- PIN is 4 binary digit

New Features:

- PIN initialisation
- PIN validation + alarm
- Update PIN

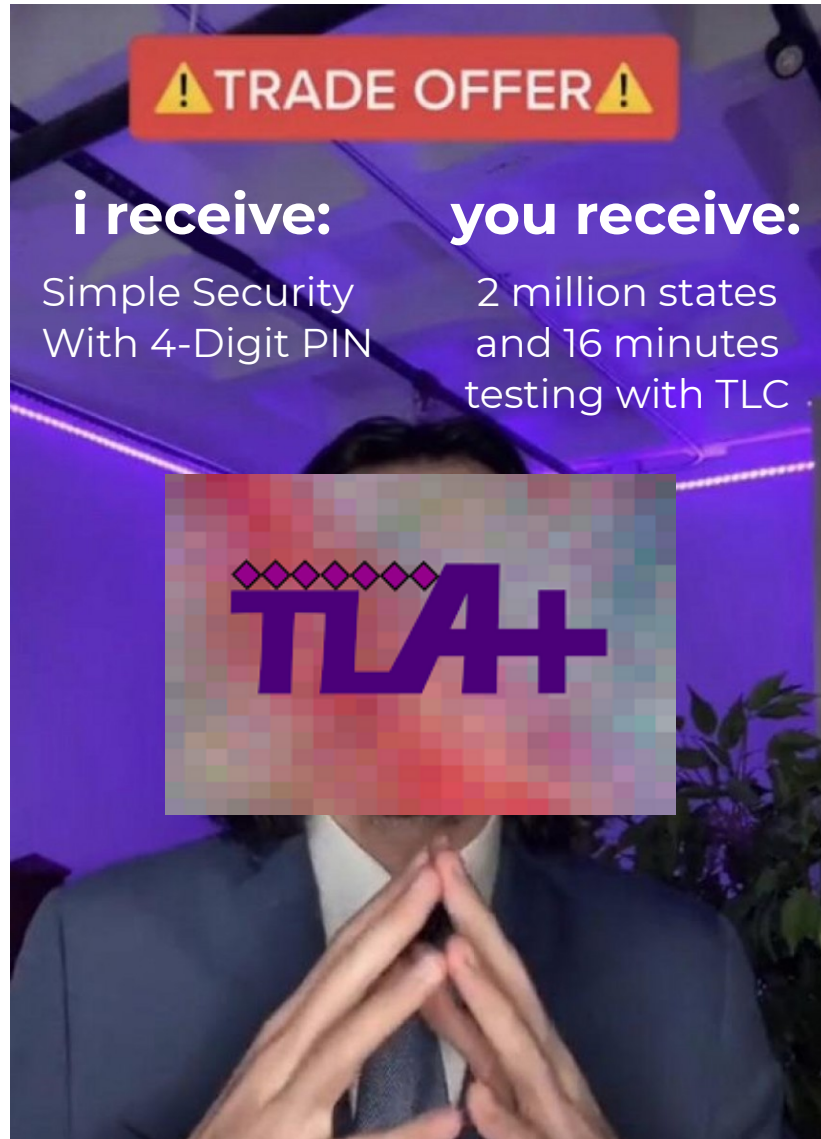
PIN Code Refinement

```

SetPinCodePC(old_pin, new_pin)  $\triangleq$ 
  Can only change PIN when car is unlocked
   $\wedge$  locked = FALSE
  Check old PIN and update accordingly
   $\wedge$  IF IsValidPin(old_pin, pin_codes)
    THEN Valid old PIN provided
       $\wedge$  pin_code_tries' = 0 Reset tries on success
       $\wedge$  pin_codes' = UpdatePinCode(old_pin, new_pin, pin_codes)
      Cancel Alarm
       $\wedge$  alarm' = FALSE
       $\wedge$  lights_on' = FALSE
       $\wedge$  RemoveTimer("armed")
       $\wedge$  RemoveTimer("alarm")
       $\wedge$  RemoveTimer("flashing_lights")
       $\wedge$  PrintT("newPinSet")
       $\wedge$  UNCHANGED {active_timers, alarm, armed, bonnet_closed,
                    closed_doors, lights_on, locked, luggage_compartment_closed,
                    luggage_compartment_locked, open_doors, timer}
    ELSE Invalid old PIN provided
       $\wedge$  pin_code_tries' = UpdateTryCount(pin_code_tries, max_pin_code_tries)
       $\wedge$  IF pin_code_tries'  $\geq$  max_pin_code_tries
        THEN  $\wedge$  StartAlarmT
               $\wedge$  UNCHANGED {armed, bonnet_closed,
                            closed_doors, locked, luggage_compartment_closed,
                            luggage_compartment_locked, open_doors, pin_codes}
            ELSE  $\wedge$  UNCHANGED {active_timers, armed, alarm, bonnet_closed,
                              closed_doors, lights_on, locked, luggage_compartment_closed,
                              luggage_compartment_locked, open_doors, timer, pin_codes}

```

PIN Code Refinement: Why Binary Codes?



00:08:07	14	1 114 660	15 328	2 420
00:09:08	15	1 244 230	17 026	2 474
00:10:10	15	1 378 564	18 617	2 510
00:11:10	16	1 491 598	19 922	2 464
00:12:12	16	1 586 570	20 946	2 360
00:13:12	17	1 690 229	22 002	2 165
00:14:13	17	1 808 935	23 091	1 719
00:15:13	18	1 903 344	23 805	1 196
00:16:15	19	1 998 611	24 356	499
00:16:46	20	2 052 624	24 512	0
00:16:48	20	2 052 624	24 512	0

PIN Code Refinement: Tested Properties

***** PROPERTIES *****

Basic Security Properties

Property 1: If locked, eventually unlock or increase attempts

$$\begin{aligned} \text{prop1} &\triangleq \text{locked} = \text{TRUE} \wedge \text{armed} = \text{TRUE} \\ &\leadsto \text{locked} = \text{FALSE} \vee \text{pin_code_tries} > 0 \end{aligned}$$

Property 2: Ensures that if the maximum tries of entering the pincode have been reached, either the correct pin is entered or the alarm is triggered

$$\begin{aligned} \text{prop2} &\triangleq \text{pin_code_tries} \geq \text{max_pin_code_tries} \\ &\Rightarrow \Diamond(\vee \text{pin_code_tries} = 0 \\ &\quad \vee \wedge \text{alarm} = \text{TRUE} \\ &\quad \wedge \text{isTimerActive}(\text{"alarm"}) \\ &\quad \wedge \text{lights_on} = \text{TRUE} \\ &\quad \wedge \text{isTimerActive}(\text{"flashing_lights"})) \end{aligned}$$

Advanced Security Properties

Property 3: PIN changes require proper authentication

$$\begin{aligned} \text{prop3} &\triangleq \\ &\text{LET } x \triangleq \text{pin_codesIN} \\ &\text{locked} = \text{FALSE} \\ &\leadsto \vee \text{pin_codes} \neq x \\ &\quad \vee \text{pin_code_tries} > 0 \\ &\quad \vee \text{locked} = \text{TRUE} \end{aligned}$$

Property 4: Under max attempts maintains normal state

$$\begin{aligned} \text{prop4} &\triangleq \text{pin_code_tries} < \text{max_pin_code_tries} \\ &\leadsto \text{locked} = \text{FALSE} \vee \text{alarm} = \text{FALSE} \end{aligned}$$

Property 5: Fresh start implies secure state

$$\begin{aligned} \text{prop5} &\triangleq \text{pin_code_tries} = 0 \\ &\leadsto \text{alarm} = \text{FALSE} \vee \text{locked} = \text{TRUE} \end{aligned}$$

State Consistency Properties

Property 6: Valid transitions for attempt counter

$$\begin{aligned} \text{prop6} &\triangleq \Box[\vee \text{pin_code_tries}' = \text{pin_code_tries} \\ &\quad \vee \text{pin_code_tries}' = \text{pin_code_tries} + 1 \\ &\quad \vee \text{pin_code_tries}' = 0]_{\text{varsPC}} \end{aligned}$$

Property 7: Failed attempts in unlocked state trigger alarm

$$\begin{aligned} \text{prop7} &\triangleq \Box(\text{alarm} = \text{TRUE} \wedge \text{locked} = \text{FALSE} \\ &\quad \Rightarrow \text{pin_code_tries} \geq \text{max_pin_code_tries}) \end{aligned}$$

Liveness Properties

Property 8: Eventually attempts reset

$$\text{prop8} \triangleq \text{pin_code_tries} > 0 \Rightarrow \Diamond(\text{pin_code_tries} = 0)$$

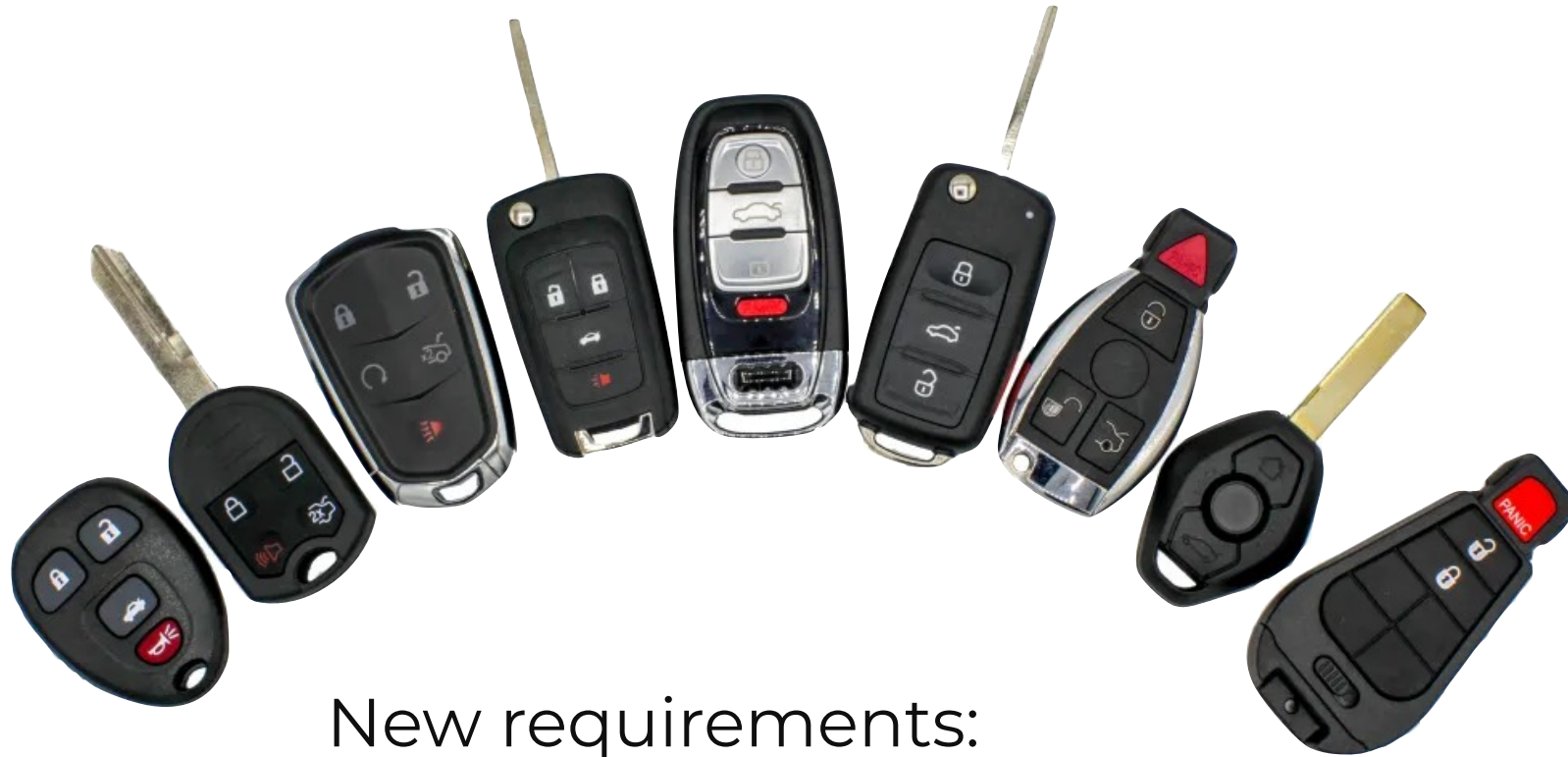
Property 9: Ensures there is always at least one valid PIN code

$$\text{prop9} \triangleq \Box(\exists x \in \text{PinCodeSet} : x \in \text{pin_codes})$$

Property 10: Alarm triggers reset attempts or maintain lock

$$\text{prop10} \triangleq \text{alarm} = \text{TRUE} \leadsto \text{pin_code_tries} = 0 \vee \text{locked} = \text{TRUE}$$

Multiple Keys Refinement



New requirements:

- Set of PINs
- Max number of PINs

Multiple Keys Refinement

MODULE *CarAlarmKeys*

New Requirements:

Requirement 8: There exists a set of keys, each with its own pincode initialized.

Requirement 9: There is a maximum number of keys.

EXTENDS *CarAlarmPincode*, *Integers*, *FiniteSets*

Constants

CONSTANT *max_number_of_keys*

ASSUME *max_number_of_keys* $\in$ *Nat*

Helper Definitions

Check if the number of keys is $\leq$ the maximum allowed

$IsValidKeySet(keys, max_keys) \triangleq Cardinality(keys) \leq max_keys$

Generate an initial set of pin codes meeting the requirements

$InitPinCodes(max_keys, pin_set) \triangleq$

LET $valid_sets \triangleq \{s \in SUBSET\ pin_set : Cardinality(s) = max_keys\}$

IN CHOOSE $k \in valid_sets$: TRUE

Initialization for car alarm system

$InitKeys \triangleq \wedge InitT$

$\wedge pin_codes = InitPinCodes(max_number_of_keys, PinCodeSet)$

$\wedge pin_code_tries = 0$

Key Invariant: Ensure type integrity and the maximum number of keys constraint

$KeyInv \triangleq IsValidKeySet(pin_codes, max_number_of_keys)$

Multiple Keys Refinement

- Builds on PIN refinement
- Keys initialisation
- Ensures max number of keys
- Note: SetPinCode can reduce number of valid PINs

***** THEOREMS *****

Theorem 1: Key invariant holds for all reachable states

THEOREM *SpecKeys*

$\Rightarrow \Box (Cardinality(pin_codes) \leq max_number_of_keys)$

Theorem 2: The car is initialized with the max number of keys

THEOREM *SpecKeys*

$\Rightarrow \Box (InitKeys \Rightarrow Cardinality(pin_codes) = max_number_of_keys)$

Distance Refinement



New requirements:

- Proximity based (un)locking
- Enable / disable proximity feature

Key features:

- Constant for enabled/disabled
- Non-deterministic action for in range / out of range

Distance Refinement: Theorems

***** THEOREMS *****

Theorem 1: If distance feature is disabled, the car can never be (un)locked by distance

THEOREM *proximity_safety_1* $\triangleq$

$$\Box(\text{key_distance_active} = \text{FALSE} \Rightarrow \\ \neg(\text{ENABLED}(\text{KeysInRangeD}) \vee \text{ENABLED}(\text{KeysOutOfRangeD})))$$

Theorem 2: If distance feature is enabled, the keys are out of range and the car is locked it's possible to become in range and unlock it

THEOREM *proximity_safety_2* $\triangleq$

$$\Box(\text{key_distance_active} = \text{TRUE} \wedge \text{key_in_range} = \text{FALSE} \wedge \text{locked} = \text{TRUE} \\ \Rightarrow (\text{ENABLED}(\text{KeysInRangeD}) \wedge \neg \text{ENABLED}(\text{KeysOutOfRangeD})))$$

Theorem 3: If distance feature is enabled, the keys are in range and the car is unlocked it's be possible to become out of range and lock it

THEOREM *proximity_safety_3* $\triangleq$

$$\Box(\text{key_distance_active} = \text{TRUE} \wedge \text{key_in_range} = \text{TRUE} \wedge \text{locked} = \text{FALSE} \\ \Rightarrow (\neg \text{ENABLED}(\text{KeysInRangeD}) \wedge \text{ENABLED}(\text{KeysOutOfRangeD})))$$

Optional Refinement: PanicMode

User triggered alarm system:

- Any time
- Hazard lights
- Alarm



Disabled via:

- Disable action
- Timer
- Unlock



Optional Refinement: PanicMode

***** THEOREMS *****

Theorem 1 : Ensures that panic mode can be triggered

THEOREM $panic_mode_activated \triangleq$
 $\Box(panic_mode = FALSE \wedge \neg(lights_on \vee alarm)$
 $\Rightarrow (ENABLED (TriggerPanicOp) \wedge \neg ENABLED (DisablePanicOp)))$

Theorem 2 : Ensures that panic mode can be disabled

THEOREM $panic_mode_disactivated \triangleq$
 $\Box(panic_mode = TRUE$
 $\Rightarrow (\neg ENABLED (TriggerPanicOp) \wedge ENABLED (DisablePanicOp)))$

Theorem 3 : Ensures that panic mode can be disabled by unlocking

THEOREM $panic_mode_unlock \triangleq$
 $\Box(panic_mode = TRUE \wedge ENABLED (UnlockOp)$
 $\leadsto panic_mode = FALSE)$

Optional Refinement: PanicMode

Status				
Checking MC_CarAlarmOptional.tla / MC_CarAlarmOptional.cfg				
Success : Fingerprint collision probability: 2.1E-6				
Start: 17:54:54 (Jan 18), end: 21:35:05 (Jan 18)				
States	Coverage			
Time	Diameter	Found	Distinct	Queue
00:00:00	0	1	1	1
00:00:03	4	40 438	1 178	1 015
00:01:03	6	534 560	8 469	5 930

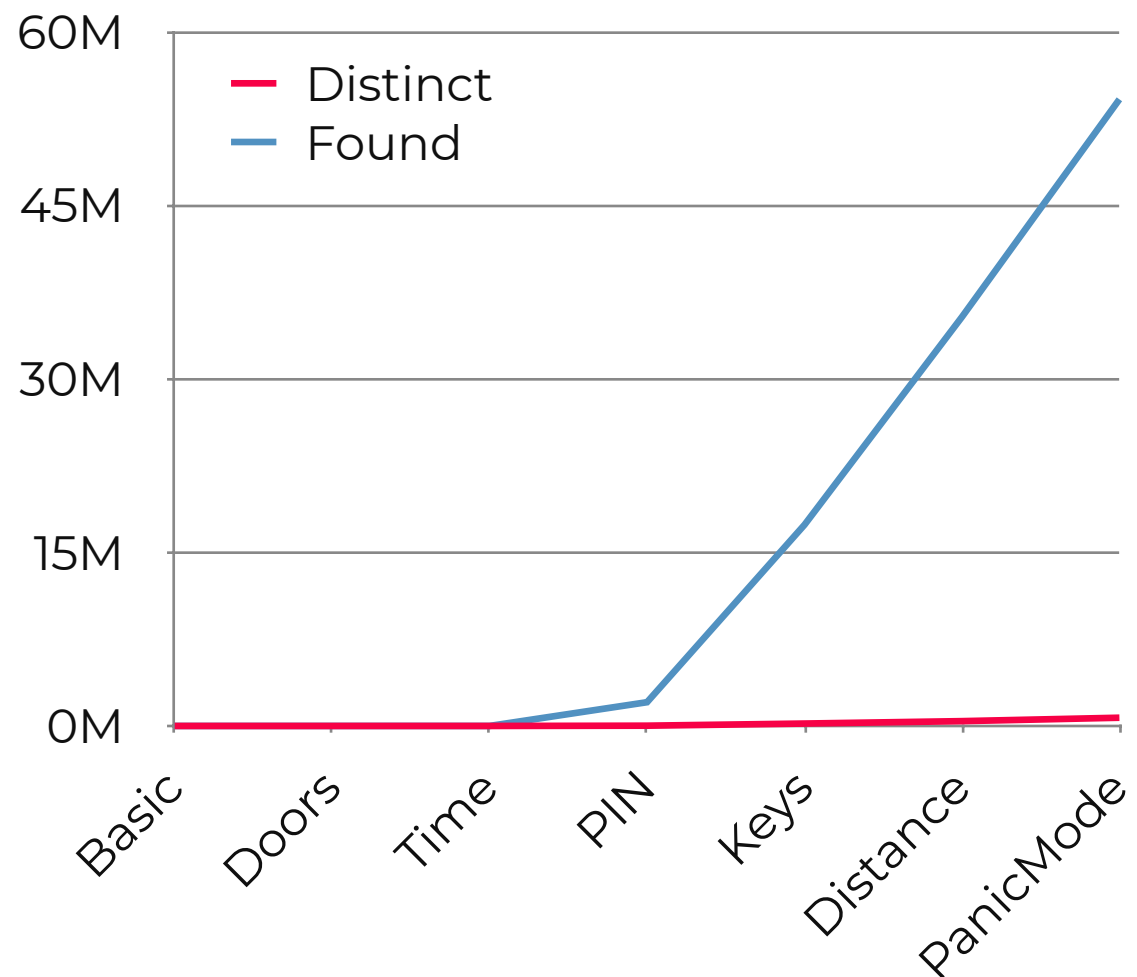


03:39:07	20	53 852 111	711 341	5 992
03:39:49	22	54 134 753	712 640	0
03:40:10	22	54 134 753	712 640	0

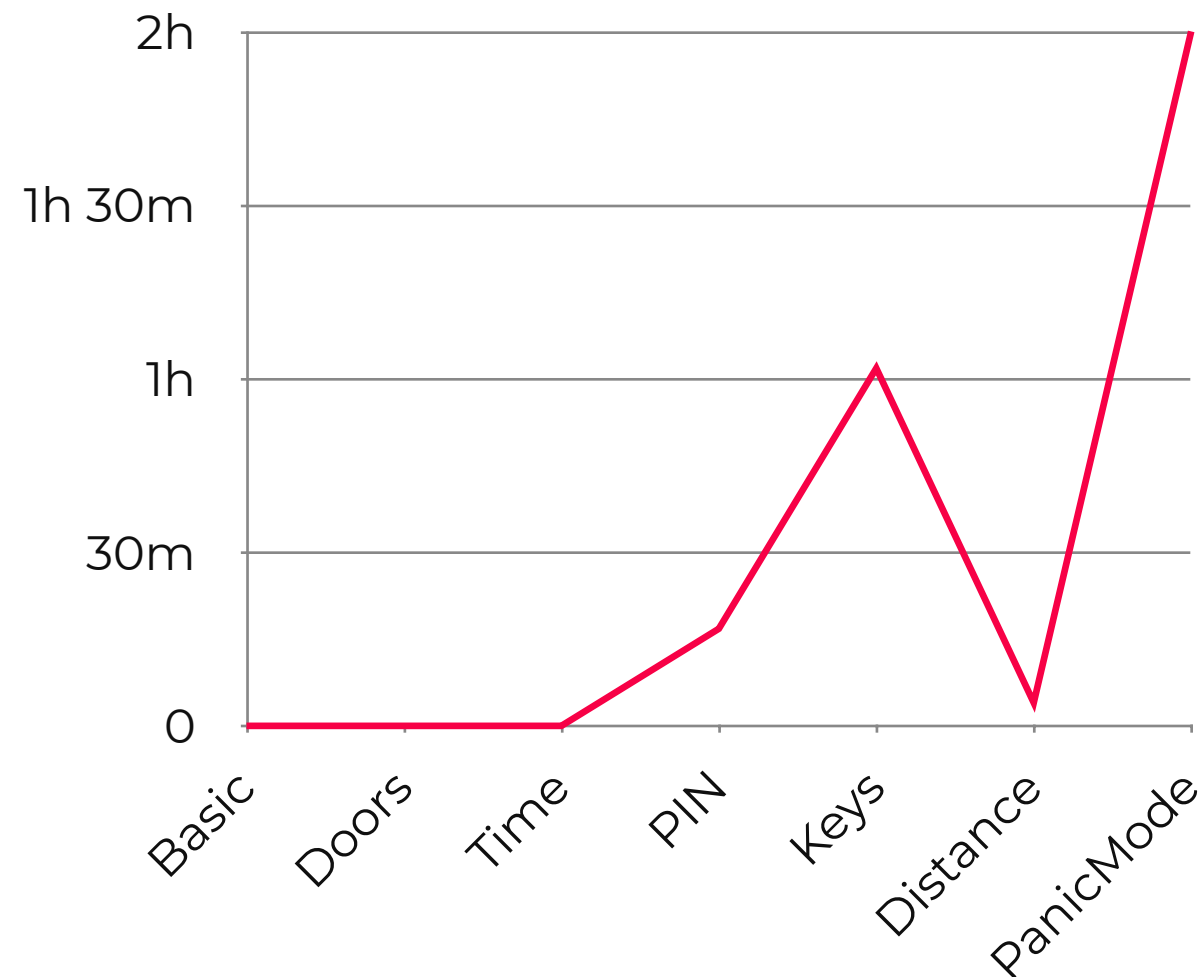
Status			
Checking MC_CarAlarmOptional.tla / MC_CarAlarmOptional.cfg			
Success : Fingerprint collision probability: 2.1E-6			
Start: 17:54:54 (Jan 18), end: 21:35:05 (Jan 18)			
States	Coverage		
Module	Action	Total	Distinct
CarAlarmOptional	InitOp	1	1
CarAlarmOptional	OpenDoorOp	478 720	15 508
CarAlarmOptional	CloseDoorOp	243 984	1 634
CarAlarmOptional	OpenBonnetOp	360 128	23 262
CarAlarmOptional	CloseBonnetOp	186 864	3
CarAlarmOptional	OpenLuggageOp	350 336	38 908
CarAlarmOptional	CloseLuggageOp	211 888	9
CarAlarmOptional	UnlockLuggageOp	279 616	45 424
CarAlarmOptional	UnlockOp	8 895 488	117 677
CarAlarmOptional	LockOp	436 288	22 712
CarAlarmOptional	TickOp	1 814 784	137 632
CarAlarmOptional	SetPinCodeOp	39 469 056	84 720
CarAlarmOptional	KeysInRangeOp	277 984	3 470
CarAlarmOptional	KeysOutOfRangeOp	218 144	0
CarAlarmOptional	TriggerPanicOp	794 240	78 608
CarAlarmOptional	DisablePanicOp	203 456	143 072

Hard Facts

Number of states per refinement



Time taken by TLC



Hard Facts

	LOC	Properties
Basic	482	8
Doors	504	10
Time	604	16
PIN	288	10
Keys	118	5
Distance	216	5
PanicMode	311	3

Always has been

Wait, it's all
PROPERTY-BASED-TESTING

?